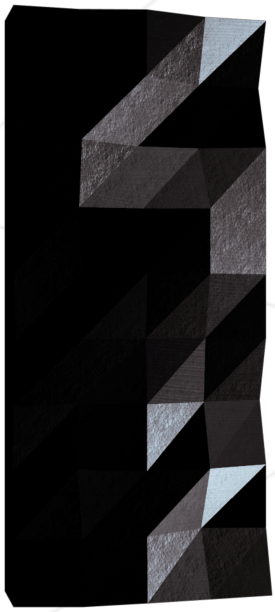

treenity

Message Queue System

Summary: Build a message queue system with topics, producers, and consumers for asynchronous message passing and event-driven communication. Store and filter data efficiently.

Version: 2.0

Contents

Chapter I

Preamble

"I write differently from what I speak, I speak differently from what I think, I think differently from the way I ought to think, and so it all proceeds into deepest darkness." - Franz Kafka

Franz Kafka's works often explore themes of alienation, bureaucracy, and the complexity of human communication. His protagonists frequently find themselves navigating labyrinthine systems where messages are delayed, misunderstood, or lost entirely. In "The Trial," Josef K. struggles to understand the charges against him through a maze of bureaucratic correspondence. In "The Castle," K. attempts to reach the castle's authorities through an intricate network of intermediaries and messages.
These literary explorations of communication systems reveal fundamental truths about how information flows through complex structures. Kafka understood that in any sophisticated system, direct communication is often impossible or inefficient. Instead, messages must be transformed, queued, and routed through various intermediaries before reaching their intended recipients. The themes of transformation, persistence, and asynchronous processing that permeate Kafka's universe reflect the very challenges that modern distributed systems must address.

Kafka's insight into the nature of indirect communication and bureaucratic systems proves remarkably prescient in our age of distributed computing and asynchronous processing.

Chapter II

Introduction

In this project, you will implement a message queue system that allows producers to publish messages to topics and consumers to subscribe and process these messages. This system enables asynchronous communication between different parts of an application or between different applications. You may choose your implementation language from the supported list provided in the annexes.
Even in small systems, components rarely move in lockstep: a signup flow emits an event, analytics listens later, and billing reacts only when it is ready. The queue becomes the shared corridor where messages wait briefly, carrying just enough context for the next consumer to continue the work.

`A message queue is a communication mechanism that allows applications to communicate by sending messages through a queue, enabling decoupled and asynchronous processing.`

II.1 Learning Objectives

- Implement IPC protocols for process communication
- Design and implement data structures for message storage
- Handle concurrent access with multiple producers and consumers
- Master concurrency patterns
- Achieve process synchronization and coordination

Chapter III

Git Workflow Guidelines

Git workflow rules are defined in the standalone document git_guidelines.pdf. All sections are mandatory. You can verify your commits and branches locally using gitinette, provided as an attachment with this subject.

Chapter IV

AI Instructions

Context

During your learning journey, AI can assist with many different tasks. Take the time to explore the various capabilities of AI tools and how they can support your work. However, always approach them with caution and critically assess the results. Whether it's code, documentation, ideas, or technical explanations, you can never be completely sure that your question was well-formed or that the generated content is accurate. Your peers are a valuable resource to help you avoid mistakes and blind spots.

Main message

- Use AI to reduce repetitive or tedious tasks.
- Develop prompting skills — both coding and non-coding — that will benefit your future career.
- Learn how AI systems work to better anticipate and avoid common risks, biases, and ethical issues.
- Continue building both technical and power skills by working with your peers.
- Only use AI-generated content that you fully understand and can take responsibility for.

Learner rules:

- You should take the time to explore AI tools and understand how they work, so you can use them ethically and reduce potential biases.
- You should reflect on your problem before prompting — this helps you write clearer, more detailed, and more relevant prompts using accurate vocabulary.
- You should develop the habit of systematically checking, reviewing, questioning, and testing anything generated by AI.
- You should always seek peer review — don't rely solely on your own validation.

Phase outcomes:

- Develop both general-purpose and domain-specific prompting skills.
- Boost your productivity with effective use of AI tools.
- Continue strengthening computational thinking, problem-solving, adaptability, and collaboration.

Comments and examples:

- You'll regularly encounter situations — exams, evaluations, and more — where you must demonstrate real understanding. Be prepared, keep building both your technical and interpersonal skills.
- Explaining your reasoning and debating with peers often reveals gaps in your understanding. Make peer learning a priority.
- AI tools often lack your specific context and tend to provide generic responses. Your peers, who share your environment, can offer more relevant and accurate insights.
- Where AI tends to generate the most likely answer, your peers can provide alternative perspectives and valuable nuance. Rely on them as a quality checkpoint.

✓ Good practice:

> I ask AI: "How do I test a sorting function?" It gives me a few ideas. I try them out and review the results with a peer. We refine the approach together.

✗ Bad practice:

> I ask AI to write a whole function, copy-paste it into my project. During peer-evaluation, I can't explain what it does or why. I lose credibility — and I fail my project.

✓ Good practice:

> I use AI to help design a parser. Then I walk through the logic with a peer. We catch two bugs and rewrite it together — better, cleaner, and fully understood.

✗ Bad practice:

> I let Copilot generate my code for a key part of my project. It compiles, but I can't explain how it handles pipes. During the evaluation, I fail to justify and I fail my project.

Chapter V

Makefile Requirements

Your project must include a Makefile that provides a standardized build interface regardless of the implementation language chosen.

Your Makefile must implement the following standard rules:

- `all`: default target producing `server` and `client`
- `clean`: remove all generated files
- `re`: execute `clean`, then `all`
- `test`: compile and run all unit tests, with appropriate exit codes

For compiled languages, generate native executables directly. For interpreted or JVM languages, generate executable shell scripts with proper shebang and argument passing.

Chapter VI

The Mission

Your mission is to build a message queue system, allowing multiple clients to communicate asynchronously. It will consist of two programs, a server and a client. They will use IPC mechanisms as the underlying transport layer.
In practice, producers emit events as they happen while consumers may lag behind or reconnect. The server keeps that flow consistent so no component has to block another to make progress.

VI.1 Terminology

- **Client**: The executable program that connects to the server
- **Producer**: A client instance that sends messages to topics
- **Consumer**: A client instance that subscribes to and receives messages from topics
- **Topic**: A named message channel that stores messages
- **Message**: A data unit consisting of a key, value, and offset
- **Offset**: A sequential identifier for messages within a topic
- **IPC Identifier**: The connection endpoint used to communicate with the server

VI.2 Server Responsibilities

The server will:

- Establish a main IPC endpoint
- Accept multiple client connections for management tasks, message production, and consumption
- Store produced messages
- Send messages to consumers

VI.3 Client Capabilities

The client executable can operate in different modes:

Management Operations

- Creating topics
- Listing available topics
- Querying client metadata

Producer Mode

- Sending messages to a specified topic
- Supporting both text and binary message formats

Consumer Mode

- Subscribing to a topic for message consumption
- Filtering messages by key prefixes
- Specifying custom starting offsets
- Maintaining consumption state across reconnections

VI.4 Message Structure

A message consists of a key, a value, and an offset. The key and value are arbitrary data, chosen by the client. The offset is an incrementing 32-bit integer, maintained by the server.
Topics and their messages are stored on the server in an in-memory data structure.

`Do not bother chunking messages into frames. If a key+body message exceeds the length limit, you may reject it (assume a 1024-byte maximum for key+body, excluding metadata).`

VI.5 Client Identification and Metadata

Each client is identified by a chosen ID. Client metadata indexing MUST use a manually implemented hashmap (standard library hashmap/dictionary types are not allowed for this part). The hashmap must implement explicit collision resolution (chaining or open addressing). The server maintains this client index with the following metadata:

- Client ID (`^[a-zA-Z0-9_.-]{1,32}$`)
- Topic to consume from (`^[a-zA-Z0-9_.-]{1,32}$`)

- Current offset
- Key prefix (optional)
- Client IPC path (consumer channel)

VI.6 Offset Management

When a client consumes a topic for the first time, it will start reading from offset 0. When receiving a message, it will commit it by responding to the server with the acknowledged offset.
When a client resumes consuming from a topic, it will start from the previously committed offset. It can also force a specific offset, to replay old messages or jump to newer ones. Optionally, a client can provide a key prefix. The server will then filter the messages, sending only the ones with a key matching the prefix. This filter should be efficient, linked to the chosen data structure.

he committed/stored offset is always the *next* topic offset the client wants to consume, not the last one it received. After processing a message delivered at topic offset N, the client commits $N + 1$. A returning subscriber with no explicit --offset resumes from that stored value. The offset shown by info follows the same convention.

VI.7 Concurrency and IPC Communication

Multiple clients can produce and listen to the same topic, and multiple servers can run separately, with no interference. The server PID will be used as a unique identifier. The main IPC endpoint should be used for management tasks (create or list topics), and for client registration. Upon completing these actions, the server will respond to the client. Since IPC mechanisms are typically one-way, you must design a method to implement bidirectional communication for responses. Each subscribed client will have its own dedicated IPC channel to receive messages.

Threading and Processing Requirements

- **Thread/Routine per Topic**: At least one dedicated thread or routine must be assigned per topic for message processing
- **Processing Model**: Message processing within a topic can be implemented as either synchronous or asynchronous, depending on your design choice
- **Concurrent Access**: Multiple threads may access the same topic data structures safely through proper synchronization

VI.8 Graceful Shutdown

When receiving SIGINT or SIGTERM, the programs should shutdown gracefully, free appropriate resources, wait for clients to commit. After a timeout of 5 seconds, it will just exit.

The server must clean up all IPC resources on stop.

A subscribed consumer's dedicated IPC channel is typically one-way (server to client), so a plain close() on the server side will not necessarily wake up a client blocked reading it. On shutdown, the server must still get every subscribed consumer to exit cleanly (exit code 0) instead of hanging. How you achieve this (a sentinel message, a dedicated signal on the channel, or any other mechanism) is your design choice, to be explained during evaluation.

VI.9 Data Structure and Prefix Matching

You may choose any efficient data structure for prefix matching (e.g., AVL tree, red-black tree, prefix tree/trie, hash table with prefix logic, or any other suitable structure). This will be discussed during the evaluation.
Libraries are tolerated for data structures, but must be justified. You are free to re-implement your own data structures if you wish. Client indexing remains mandatory: use your own hashmap implementation for client metadata.

Data Structure Indexing

- **Client Indexing Requirement**: Client metadata indexing requires a manually implemented hashmap with collision resolution (chaining or open addressing)
- **Consumer Indexing**: Topic handlers maintain an efficient data structure of consumers indexed by prefix
- **Efficient Lookup**: Fast retrieval of all consumers matching a message key prefix
- **Multiple Consumers**: Multiple consumers can share the same prefix

Prefix Matching Rules

- **Exact Match**: `key_prefix = "user"` matches `message_key = "user"`
- **Prefix Match**: `key_prefix = "user"` matches `message_key = "user.login"`
- **No Match**: `key_prefix = "user"` does NOT match `message_key = "admin"`

- **Wildcard**: `key_prefix = ""` (empty) matches all messages
- **Direct Addition**: Empty-prefix consumers are added directly to the delivery list without data structure traversal

VI.10 Offset Behavior

1. **New Consumer**: Starts at offset 0 (beginning of topic)
2. **Returning Consumer**: Resumes from last saved offset
3. **Custom Offset**: Consumer provides `requested_offset` in registration or resuming
4. **Offset Updates**: Server updates consumer offset after client has acknowledged

VI.11 Unit Testing Requirements

The prefix matching functionality requires comprehensive unit testing:

- **Basic Prefix Matching**: Test exact matches, partial matches, and non-matches
- **Empty Prefix Handling**: Verify empty prefix consumers receive all messages
- **Data Structure Operations**: Test correctness of your chosen data structure's operations and lookups
- **Edge Cases**: Test with empty data structures, minimal datasets, and large-scale configurations
- **Integration**: Test complete filtering algorithm with mixed prefix types

VI.12 Signal Handling Details

- **SIGINT (Ctrl+C)**: Triggers graceful disconnection process
- **SIGTERM**: Also triggers graceful disconnection
- **Cleanup**: Consumer sends disconnect message to server before exit, and server cleans up

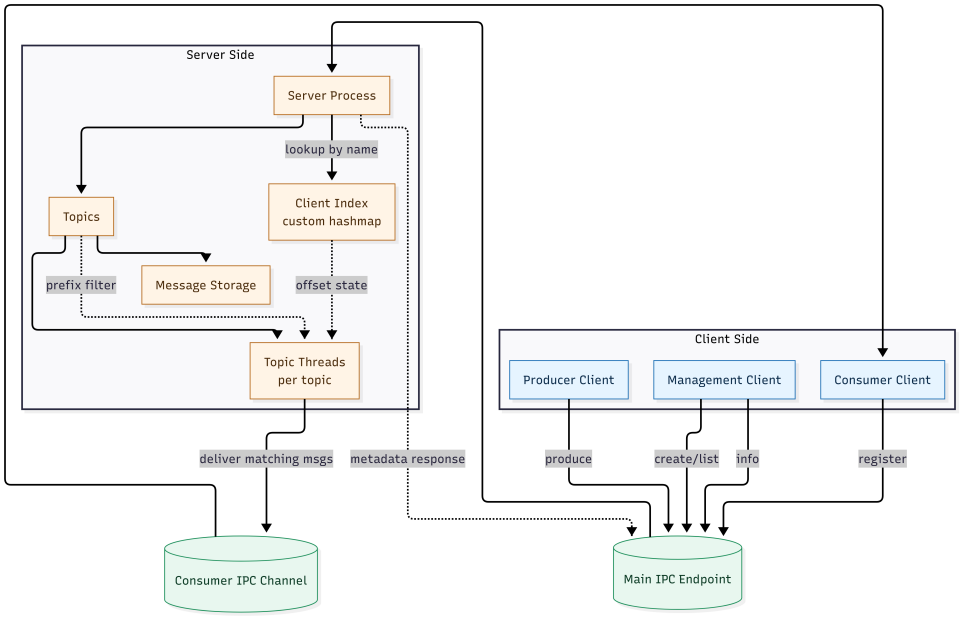

Figure VI.1: Expected project architecture.

Chapter VII

IPC Implementation Choice

For the inter-process communication mechanism, you must choose from message queue mechanisms:

- **System V Message Queues**: All related functions are authorized
- **POSIX Message Queues**: All related functions are authorized
- **Named Pipes (FIFOs)**: All related functions are authorized

During the evaluation, you must be able to explain and justify your choice of IPC implementation. Be prepared to discuss the advantages, limitations, and specific use cases of your chosen mechanism in relation to message queue design and your implementation approach.

Chapter VIII

Commands Specification

This chapter defines the command-line interface for your message queue system. All executables must implement these exact command formats and behaviors.

CRITICAL: Your implementation will be tested with automated test binaries that rely on exact command-line arguments, stdin/stdout behavior, and exit codes. Some test binaries are provided with this subject for development testing, and additional test binaries will be provided during evaluation. Any deviation from these specifications will cause test failures. The command-line interface must work exactly as specified - argument order, output format, and behavior must match precisely.

VIII.1 Core Commands

Server

```
./server
```

- **Purpose**: Central management process
- **Function**: Handles client connections and manages topics
- **Output (stdout):** Starting message with IPC identifier (e.g., "/tmp/operator.server.1337")
- **Signal handling**: SIGINT/SIGTERM for graceful disconnection

Client

The client executable provides all topic management, producer, and consumer functionality through subcommands.

Topic Creation

```
./client <ipc_identifier> create <topic_name>
```

- **Purpose**: Create a new topic
- **Arguments**: IPC identifier from server output, topic name
- **Output (stdout):** "topic created" confirmation message

Topic Listing

```
./client <ipc_identifier> list
```

- **Purpose**: List all available topics
- **Arguments**: IPC identifier from server output
- **Output (stdout):** Comma-separated topic names (e.g., "user_events,orders")

Producer (Produce Messages)

```
./client <ipc_identifier> produce <topic_name> [--raw]
```

- **Purpose**: Send messages to a topic
- **Input**: Text format (`key:body`) from stdin, or binary format with `--raw`
- **Arguments**: IPC identifier, target topic name
- **Options**: `--raw` for binary message format

If topic_name does not exist, the producer must exit with code 2 (see Exit Codes) instead of silently accepting messages for it. How and when you detect this is your design choice: an initial handshake with the server, a preliminary existence check, or anything else that gets you there.

Consumer (Subscribe to Messages)

```
./client <ipc_identifier> subscribe <topic_name> <subscriber_name> [--prefix <prefix>] [--offset <offset
    >] [--raw]
```

- **Purpose**: Subscribe to receive messages from a topic
- **Arguments**: IPC identifier, topic name, unique client identifier
- **Options**:
  - `--prefix` to filter messages by key prefix
  - `--offset` to specify starting message position (defaults to offset 0 for new consumers)
  - `--raw` for binary output format

- **Output (stdout):** "subscribed to <topic>" confirmation, followed by matching messages
- **Signal handling**: SIGINT/SIGTERM for graceful disconnection

A returning subscriber can specify a new topic, offset or prefix. Two subscribers with the same name cannot run at the same time.

Client Metadata (Info)

```
./client <ipc_identifier> info <subscriber_name>
```

- **Purpose**: Query server-side metadata for a registered client
- **Arguments**: IPC identifier from server output, client identifier
- **Output (stdout):** JSON object with `client`, `topic`, `offset`, `prefix`, `ipc`

VIII.2 Message Formats

Text Mode (Default)

Producer Input:

```
key:body
key:body
```

Consumer Output:

```
key:body
key:body
```

Raw Binary Mode (--raw flag)

Producer Input:

```
[keysize:int32][key:bytes][valuesize:int32][value:bytes]
```

Consumer Output:

```
[offset:int32][keysize:int32][key:bytes][valuesize:int32][value:bytes]
```

The keysize and valuesize fields are binary-encoded 32-bit integers (4 bytes each), not string representations. Use little-endian byte order consistently across your implementation.

Records are written back-to-back with no separator and no padding between them. EOF must always land on a record boundary; a partial record at EOF is a producer error (exit code 1, message to stderr). There is no maximum number of records per invocation.

VIII.3 Example Usage

Server Startup

```
./server
```

Output (stdout):

```
/tmp/operator.server.1337
```

Topic Management

```
./client /tmp/operator.server.1337 create user_events
```

Output (stdout):

```
topic created
```

```
./client /tmp/operator.server.1337 list
```

Output (stdout):

```
user_events,orders
```

Consumer Subscription

```
./client /tmp/operator.server.1337 subscribe user_events client0 --prefix user.create &
```

Output (stdout):

```
subscribed to user_events
```

Message Production and Consumption

```
./client /tmp/operator.server.1337 produce user_events
```

Input (stdin):

```
user.create:{"user":"reach","age":25}
user.update:{"user":"norminet","age":42}
user.delete:{"user":"froz","age":24}
user.create:{"user":"moulinette","age":1337}
```

Output (stdout) from subscribed client:

```
user.create:{"user":"reach","age":25}
user.create:{"user":"moulinette","age":1337}
```

Client Metadata

```
./client /tmp/operator.server.1337 info client0
```

Output (stdout), after the two messages above were committed:

```
{"client":"client0","topic":"user_events","offset":4,"prefix":"user.create","ipc":"/tmp/operator.server
    .1337.client0"}
```

```
$> ./server &
[1] 2645758
/tmp/treenity.server.2645758

$> ./client /tmp/treenity.server.2645758 create user_events
topic created

$> ./client /tmp/treenity.server.2645758 list
user_events

# in a second terminal, subscribe in the background
$> ./client /tmp/treenity.server.2645758 subscribe user_events client0 --prefix user.create &
[2] 2645910
subscribed to user_events

# back in the first terminal, produce four messages on stdin
$> ./client /tmp/treenity.server.2645758 produce user_events
user.create:{"user":"reach","age":25}
user.update:{"user":"norminet","age":42}
user.delete:{"user":"froz","age":24}
user.create:{"user":"moulinette","age":1337}

# client0 (second terminal) received only the matching prefix
user.create:{"user":"reach","age":25}
user.create:{"user":"moulinette","age":1337}

$> ./client /tmp/treenity.server.2645758 info client0
{"client":"client0","topic":"user_events","offset":4,"prefix":"user.create","ipc":"/tmp/treenity.server.2645758.client0"}

$>
```

Figure VIII.1: The full example above, captured from a real run against the reference solution.

All program outputs specified in this document (IPC identifiers, confirmations, topic lists, messages) must be written to stdout for test program parsing. Human-readable logs, debug information, and error messages should be written to stderr. Only stdout outputs will be evaluated by automated testing tools.

VIII.4 Error Handling and Exit Codes

Your implementation must handle error conditions gracefully and return appropriate exit codes:

Exit Codes

- **0**: Success
- **1**: General error (invalid arguments, connection failure)
- **2**: Topic error

- **3**: IPC communication error

Error Conditions

- **Invalid IPC identifier**: Exit with code 1, write error message to stderr
- **Topic does not exist**: For consume/produce operations, exit with code 2, write error message to stderr
- **Duplicate topic creation**: Exit with code 2, write error message to stderr
- **Client not found**: For metadata queries, exit with code 2, write error message to stderr
- **Invalid client name format**: Exit with code 1, write error message to stderr
- **Duplicate client name**: Exit with code 2, write error message to stderr
- **Server disconnection**: Exit with code 3, attempt graceful cleanup, write error message to stderr

If more than one error condition applies at once, exit codes are evaluated in this order of precedence: 1 (general/args/IPC-identifier) > 3 (IPC communication) > 2 (topic/client). The first applicable code is returned.

VIII.5 Message Ordering and Concurrency

Message Ordering Guarantees

- **Per-Producer Ordering**: Messages from a single producer are stored in the order they are received
- **Cross-Producer**: No ordering guarantees between different producers
- **Consumer Delivery**: Messages are delivered to consumers in offset order

VIII.6 Test Scenario

An archive containing two helper binaries is provided with this subject. These tools can be used to validate your implementation and will also be used during evaluation.

- **ft_aquarium**: An event-driven aquarium visualizer that connects to a topic as a consumer and renders incoming messages as fish.
- **ft_fish**: producer application that generates random fish messages and publishes them to the aquarium topic.

Both binaries support the `-h` option, which displays their available command-line arguments and usage instructions.

Chapter IX

Readme Requirements

A `README.md` file must be provided at the root of your Git repository. Its purpose is to allow anyone unfamiliar with the project (peers, staff, recruiters, etc.) to quickly understand what the project is about, how to run it, and where to find more information on the topic.
The `README.md` must include at least:

- The very first line must be italicized and read: *This project has been created as part of the 42 curriculum by <login1>[, <login2>[, <login3>[...]]].*
- A "**Description**" section that clearly presents the project, including its goal and a brief overview.
- An "**Instructions**" section containing any relevant information about compilation, installation, and/or execution.
- A "**Resources**" section listing classic references related to the topic (documentation, articles, tutorials, etc.), as well as a description of how AI was used — specifying for which tasks and which parts of the project.

➠ **Additional sections may be required depending on the project** (e.g., usage examples, feature list, technical choices, etc.).

Any required additions will be explicitly listed below.

For this project, the `README.md` must also include:

- An "**Architecture**" section explaining your server design choices: the threading/routine model per topic and the synchronization strategy used to protect shared state.
- An "**IPC Choice**" section justifying the chosen mechanism (System V message queues, POSIX message queues, or Named Pipes) and explaining how bidirectional communication was achieved.
- A "**Data Structures**" section describing your client metadata hashmap implementation and the prefix-matching structure you chose, with justification.

- A “**Testing**” section explaining how the prefix-matching unit tests are built and run.

```
Your README must be written in English.
```

Chapter X

Annexes

X.1 Supported Languages

You may implement this project in one of the following languages:

- **C++** (see the C++ Implementation Guidelines section)
- **Go** (see the Go Implementation Guidelines section)

All implementations must use one of the authorized IPC mechanisms for inter-process communication, as discussed in the IPC Implementation Choice section.

X.2 C++ Implementation Guidelines

Concurrency Patterns

- Use `std::thread` or thread pools for concurrent processing
- Implement at least one dedicated thread per topic for message processing
- Implement synchronization with `std::mutex` and `std::condition_variable`
- Consider `std::shared_mutex` for reader-writer patterns
- Use RAII for resource management and cleanup

```
Compile with C++17 or later: std::shared_mutex needs it. Your
Makefile must set CXXFLAGS accordingly.
```

IPC Integration

- Use standard library functions if available for your chosen IPC mechanism
- External IPC libraries are not allowed
- Handle proper message structure and communication management
- Ensure proper cleanup and resource management

External Libraries

- External libraries are allowed if they do not solve an entire section of the subject
- Example: a lightweight argument parsing library such as CLI11 or cxxopts is acceptable

Unit Testing Framework

- **Required**: Use Google Test (gtest) framework for unit testing
- **Requirements**: All prefix matching functionality must have comprehensive unit tests
- **Integration**: Tests must be buildable through Makefile with a `test` target

X.3 Go Implementation Guidelines

Concurrency Patterns

- Use goroutines for handling concurrent clients
- Implement at least one dedicated goroutine per topic for message processing
- Use Go channels for internal message routing and communication between goroutines
- Utilize `sync.RWMutex` for topic-level data synchronization when needed
- Consider `context.Context` for request cancellation and timeouts

IPC Integration

- Use standard library functions if available for your chosen IPC mechanism
- If no standard library support exists, use CGo to call IPC functions
- External IPC libraries are not allowed
- Handle proper error conversion and resource management
- Manage IPC resource lifecycle appropriately

External Libraries

- External libraries are allowed if they do not solve an entire section of the subject
- Example: Cobra for argument parsing is acceptable

Unit Testing Framework

- **Required**: Use Go's native `testing` package with `go test`
- **Requirements**: All prefix matching functionality must have comprehensive unit tests
- **Integration**: Tests must be runnable through Makefile with a `test` target

Chapter XI

Submission and peer-evaluation

Turn in your assignment in your `Git` repository as usual. Only the work inside your repository will be evaluated during the defense. Don't hesitate to double-check the names of your files to ensure they are correct.

Recode instructions

During the evaluation, a brief **modification of the project** may occasionally be requested. This could involve a minor behaviour change, a few lines of code to write or rewrite, or an easy-to-add feature.

While this step may **not be applicable to every project**, you must be prepared for it if it is mentioned in the evaluation guidelines.

This step is meant to verify your actual understanding of a specific part of the project. The modification can be performed in any development environment you choose (e.g., your usual setup), and it should be feasible within a few minutes — unless a specific time frame is defined as part of the evaluation.
You can, for example, be asked to make a small update to a function or script, modify a display, or adjust a data structure to store new information, etc.

The details (scope, target, etc.) will be specified in the **evaluation guidelines** and may vary from one evaluation to another for the same project.